Table of Contents

1 · Executive Summary **1**

2 · Product Definition and Gong Parity **2**

3 · System Architecture **4**

4 · Hybrid AI Engine (Cloud and Local) **6**

5 · Data Model **10**

6 · Feature Specifications - Four Pillars **12**

Pillar 1 - The Core Loop 13

Pillar 2 - Deal Intelligence 13

Pillar 3 - Coaching 14

Pillar 4 - The Library 14

7 · Design System - Warm Calm **14**

8 · Security, Privacy and Legal **16**

9 · Build Roadmap and Risk Register **17**

10 · Appendix - Hardware, Models, Costs, Glossary **18**

1 · Executive Summary

AMBER is a private, single-user revenue intelligence platform - a personal, better-than-Gong built for exactly one operator: you. Gong.io earns its reputation by recording sales conversations, transcribing them, and mining every call for coaching signals, deal risk, and pipeline truth. It is also an enterprise product: publicly reported 2026 pricing analyses put a single seat at roughly $1,360 to $1,600 per year on top of a $5,000+ annual platform fee and about $7,500 in onboarding, typically locked into multi-year terms. There is no personal tier, and there never will be - the economics of an enterprise sales motion cannot shrink to one user. This blueprint specifies the system that gives you the same capability class without any of that overhead.

The design thesis is simple: for a single user with a laptop, the constraints that shape Gong fall away. You do not need multi-tenant isolation, CRM synchronization, SSO, or a sales-ops dashboard. You need a fast ingestion pipeline, honest transcripts, insightful analysis, and a search engine over your own conversation history. Every architectural decision in this document follows from that reduction. The result is a Next.js monolith with an embedded job pipeline, a hybrid AI engine that runs local models by default and escalates to cloud APIs only when a task genuinely benefits, and a Warm Calm interface that makes reviewing your own calls feel pleasant rather than clinical.

$0	100%	4	5
per-seat cost, forever	offline-capable by default	feature pillars	AI provider adapters

Four pillars were confirmed as in-scope at project kickoff, and together they cover the full Gong capability surface at personal scale. The Core Loop pillar handles the path from a raw recording to a clean transcript, computed conversational metrics, structured insights, meeting notes, and a follow-up email draft. Deal Intelligence extracts MEDDIC fields, raises risk flags, and maintains a local pipeline board with an auto-built deal timeline. Coaching scores every call against a rubric you own, tracks your trends, and curates a best-practice library from your own winning moments. The Library pillar gives you semantic search across every transcript you have ever uploaded, with filters and highlights.

The hybrid AI engine is the piece that makes this better rather than merely cheaper. Every AI task in AMBER - transcription, summarization, deep analysis, field extraction, scoring, embedding - is declared with a policy: which local model class handles it by default, under what conditions it escalates to a cloud provider, and which cloud chain serves as fallback. Local-first means your audio never leaves your machine; cloud escalation only ever sees text, optionally redacted, and only when the policy allows it. This is the architecture Gong cannot offer you, because it is the architecture only a personal system can afford.

Decision	Choice	Rationale
Application shape	Next.js 16 monolith (App Router, TypeScript)	One process, one command, zero deployment story; personal scale removes every reason for microservices
Database	SQLite via Prisma	Single-file, zero-ops, trivially backed up; hundreds of calls per year is three orders of magnitude below any pressure point
Local LLM runtime	Ollama (OpenAI-compatible endpoint)	One binary, one REST API, model pulls on demand; endpoint compatibility means one client covers local and cloud
Local speech-to-text	whisper.cpp bindings, faster-whisper sidecar as alternative	Whisper-class accuracy offline; faster-whisper is ~4x faster than reference Whisper at equal accuracy
Cloud providers	OpenAI, Anthropic, Gemini, any OpenAI-compatible endpoint	Vendor choice per task, per day; no lock-in, and one adapter family covers Groq, DeepSeek, Together and friends
Media storage	Content-addressed files on disk	Recordings referenced by SHA-256; the database stays tiny and restorable independently of bulky audio

Table 1 - Key architecture decisions at a glance

Success for v1 is defined narrowly and honestly. A recording dragged into the inbox produces, within one coffee, a speaker-attributed transcript, a talk-ratio you can trust because it is computed arithmetically rather than guessed by a model, a set of structured insights with confidence scores, extracted MEDDIC fields on the linked deal, a scorecard, and a follow-up email draft. Everything runs offline on a mid-range machine. Cloud is opt-in per policy, never a silent dependency. If any of those steps requires you to open a terminal, the build is not done.

2 · Product Definition and Gong Parity

Gong.io is a revenue intelligence platform. Its product surface, reduced to the parts that matter to a single operator, covers five layers: capture (recording calls through calendar bots and platform integrations), transcription (speaker-attributed text with timestamps), conversation intelligence (talk ratios, topics, objections, sentiment, keywords), revenue intelligence (deal tracking, risk flags, forecast impact), and coaching (scorecards, best-practice libraries, trend analytics). Enterprise features - team leaderboards, CRM sync, forecasting committees, permission matrices - are deliberately out of scope for AMBER, not because they are hard, but because they are meaningless for one user.

The parity table below is the contract for what AMBER v1 must match, exceed, or consciously skip. The general rule applied throughout: AMBER must exceed Gong wherever personal scale is an advantage (privacy, customizability, prompt transparency, cost), match Gong wherever the capability is algorithmic

(metrics, search), and skip Gong wherever the feature exists only to serve an organization (multi-user anything). Where a capability degrades at personal scale - for example, "top performer" benchmarks, which need a team - AMBER replaces it with something a solo operator can actually use: comparison against your own best calls.

Gong capability	AMBER v1	Approach
Calendar bot capture (Zoom/Meet/Teams)	Skip	You chose manual upload; a bot is an enterprise artifact with legal and reliability costs - revisit in v2 only if upload friction actually hurts
Transcription + diarization	Match	Whisper-class local STT; pause-based speaker clustering plus one LLM relabel pass; word timestamps via the alignment upgrade path
Talk-listen ratio, pacing, question rate	Exceed	Computed deterministically from segment timings - no model in the loop, so the numbers are reproducible and auditable
Topics, objections, sentiment, keywords	Match	Structured-JSON extraction prompts; confidence-scored; every prompt visible and editable in Settings
Deal board + risk flags	Exceed	MEDDIC auto-extraction, no-next-step and single-threading detection; the board is local, fast, and yours
Best-practice library	Exceed	Curated from your own winning calls via embeddings; in Gong this needs a team, here it needs only history
Team leaderboards / CRM sync / SSO	Skip	Organizationally meaningless for one user; their absence is a feature - it is why the stack stays simple
Forecasting	Skip	A forecast of one person's pipeline by one person adds no information; the deal board already shows the truth

Table 2 - Gong parity matrix for AMBER v1

The persona is a solo seller - a founder, an independent rep, or a consultant whose income depends on how well their calls go. Their week contains perhaps five to fifteen recorded conversations. Their pain is not volume; it is that every call evaporates the moment it ends. Notes are forgotten, objections recur unhandled, the same discovery questions go unasked for weeks, and nobody - least of all a manager, because there is none - ever replays the tape. AMBER exists to be that replay: a patient, private analyst that reads every call and never forgets any of them.

- Non-goal: any multi-user surface - roles, sharing, leaderboards, or permissioning. One user, one login-free local app.
- Non-goal: monetization. This is a personal system; no telemetry, no accounts, no licensing of your data to anyone.
- Non-goal: real-time or live capture. AMBER is post-call by design; you deliberate over recordings, not streams.

- Non-goal: mobile. The work happens at a desk after the call; a small-screen version would compromise the transcript-reading experience.

3 · System Architecture

The architect's decision, as delegated at kickoff, is a single Next.js 16 monolith: one Node process serving the UI, the API routes, and the processing pipeline, with SQLite as the only database and the file system as the media vault. This is not a compromise - at personal scale it is strictly superior. A distributed design would add a deployment story, a network story, and a failure story, and would buy nothing in exchange. The monolith starts with one command, backs up with one copy, and fails in ways one person can debug. Every layer is swappable later without regret, because the boundaries that matter (provider adapters, the pipeline stages, the data model) are module boundaries, not network boundaries.

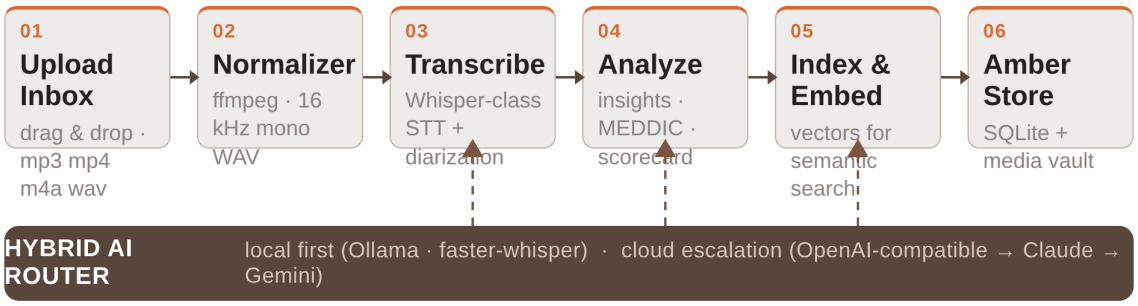


Figure 1 - The AMBER processing pipeline. Solid arrows carry data left to right; the dashed risers show the Hybrid AI Router serving the three AI-consuming stages.

The pipeline is a six-stage flow over a call record. The Upload Inbox stage accepts a drag-andropped file (mp3, mp4, m4a, wav, or webm), hashes it with SHA-256 for identity and de-duplication, and enqueues a job. The Normalizer strips video, resamples to 16 kHz mono WAV with ffmpeg, and applies loudness normalization - a step that measurably improves Whisper-class accuracy on quiet recordings. Transcribe runs the local STT engine and emits timestamped segments; diarization is folded into this stage as a clustering post-pass, with one LLM call relabeling clusters into readable speaker names. Analyze is the insight engine: a small set of structured-output prompts producing topics, objections, sentiment, MEDDIC fields, risk flags, the scorecard, notes, and the follow-up draft. Index and Embed produces the vectors that power semantic search. The Amber Store persists everything - the database row tree plus the media vault.

Layer	Choice for v1	Considered and deferred
Runtime	Next.js 16 App Router, TypeScript, Node 22	Electron/Tauri shell - revisit only if browser chrome ever annoys in daily use
Database	SQLite + Prisma ORM	Postgres - unjustified ops cost at this scale; the schema ports cleanly if ever needed
Job queue	DB-backed jobs table with an in-process worker loop	Redis + BullMQ - a second process and a second datastore to babysit, for zero observable gain
Audio tooling	ffmpeg via ffmpeg-static	Native decoding - no; ffmpeg is the universal solvent and a static binary ships with the app
STT engine	whisper.cpp through Node bindings	faster-whisper sidecar (a ~40-line FastAPI service) as the documented alternative; both expose the same interface
Embeddings	nomic-embed-text via Ollama	mxbai-embed-large if retrieval quality disappoints; brute-force cosine first, sqlite-vec only if the library grows past thousands of calls
Vector store	In-database vectors, brute-force cosine	A vector database is absent until personal scale produces a problem it would solve

Table 3 - Component choices and their alternatives

Two architectural guarantees are worth stating explicitly because they shape everything downstream. First, the pipeline is resumable: every stage writes its output to the call record before advancing, so a crash or a closed laptop costs at most one stage of one call, and re-running is idempotent by design. Second, the pipeline is provider-blind: stages declare what they need (a transcript, an embedding, a structured extraction) and the router decides who serves it. No stage contains a URL, a model name, or an API key. That single discipline is what makes the hybrid engine in Chapter 4 possible, and it is also why a new cloud provider is a 100-line adapter rather than a refactor.

Concurrency is deliberately boring. The worker processes one call at a time through the stages, and within a call, stages run sequentially. A 30-minute call transcribes in a few minutes on a typical machine - the queue exists so that dropping three recordings after a call block just works, in order, without contention for the GPU or CPU that local inference prefers. There is no parallelism story in v1 because there is no problem for it to solve; the design leaves room for a second worker lane gated on a hardware check, but it ships switched off.

4 · Hybrid AI Engine (Cloud and Local)

The hybrid engine is AMBER's heart, and it exists to satisfy one rule you set at kickoff: the system must run on both cloud APIs and local models, and it must prefer local unless there is a reason. The engine is built as three cooperating pieces. A provider registry knows every adapter - local and cloud - along with its capabilities, costs, and health. A per-task policy table, editable in Settings, decides for each AI task which tier serves it by default and when escalation is permitted. A router executes the policy: attempt, validate, escalate on failure or policy trigger, and log every run for cost and latency telemetry.

The provider abstraction is intentionally narrow. Every adapter implements four operations - chat, structured chat (JSON-schema constrained output), embed, and transcribe - plus a health check. Five adapters ship in v1: Ollama (local, OpenAI-compatible REST), OpenAI, Anthropic, Gemini, and a generic OpenAI-compatible adapter that covers Groq, DeepSeek, Together, LM Studio, or a personal proxy by URL and key alone. Because Ollama speaks the OpenAI wire format, the local and generic-cloud adapters share nearly all their code, which is exactly the kind of leverage a small system should extract from a lucky standard.

Provider	Type	Default role in AMBER
Ollama (localhost:11434)	Local	Default chat, structured extraction, and embeddings; serves every task that does not explicitly demand cloud
OpenAI-compatible (custom)	Cloud	First escalation tier for analysis-class tasks; also the shape used for Groq, DeepSeek, Together, LM Studio, proxies
Anthropic	Cloud	Long-context deep analysis; a 200K-class window swallows a two-hour transcript in one pass
OpenAI	Cloud	Premium analysis tier and cloud STT (Whisper API) when local transcription is not available
Gemini	Cloud	Budget-friendly analysis chain and multimodal experiments; generous free tier suits a personal system

Table 4 - Provider registry as configured by default

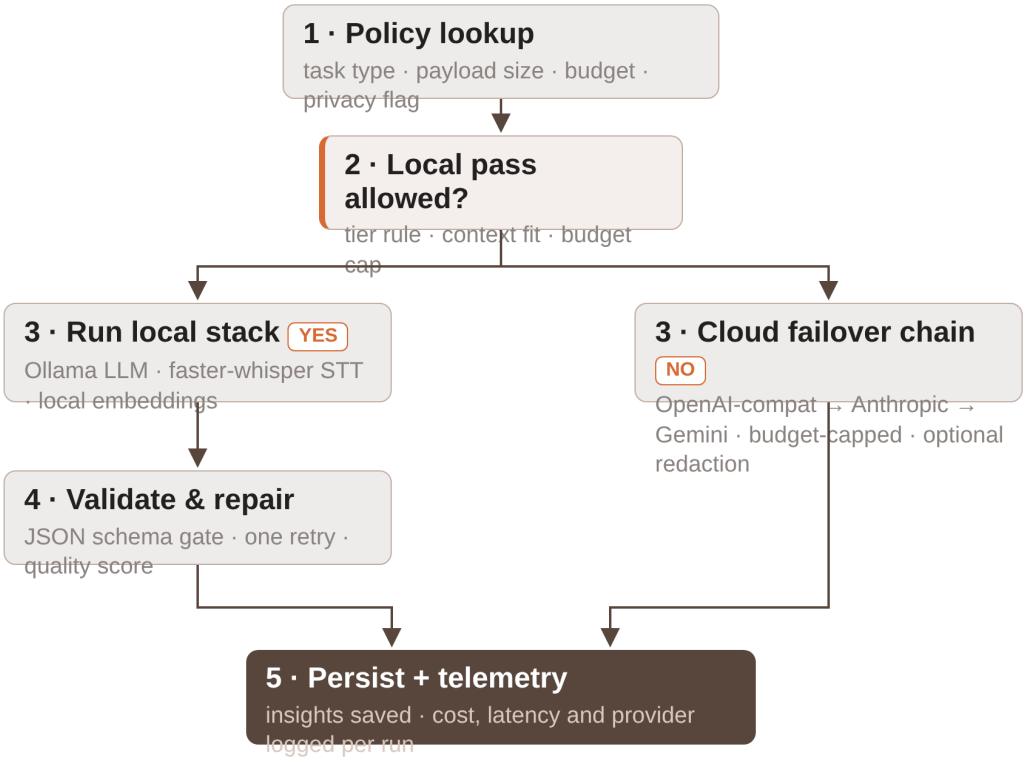


Figure 2 - Routing decision flow. The local path is attempted first under policy; validation failures and policy triggers send the task down the cloud failover chain, and every run is persisted with telemetry.

Escalation triggers are enumerated, not vibes. A task escalates when: the payload exceeds the local model's effective context; the task's policy tier is set to cloud-only; the local attempt fails validation twice (a malformed JSON schema, an empty extraction, a quality score below threshold); the monthly cloud budget has room and the task is marked cost-insensitive; or you flipped the per-call "deep analysis" switch. Conversely, cloud never sees audio: STT runs locally by default, and when text goes to a cloud provider an optional redaction pass first strips names, emails, phone numbers, and any custom patterns you define. The redacted map is kept locally so insights can be re-identified on your screen only.

Task	Local default	Escalates when	Cloud fallback chain
Transcription (STT)	whisper.cpp / faster-whisper, small-or-better	Local engine absent; user sets cloud STT	Whisper API (OpenAI)
Diarization relabel	Local 7-8B class LLM	Never in v1	n/a (heuristic output stands)
Summary + meeting notes	Local 7-8B class	Context overflow only	OpenAI-compat, then Anthropic
Deep call analysis	Local attempt first	Quality gate fails; deep switch on	OpenAI-compat, Anthropic, Gemini
MEDDIC extraction	Local 7-8B, structured JSON	Validation fails twice	Anthropic (200K), then OpenAI-compat
Objection + sentiment pass	Local 7-8B, structured JSON	Quality gate fails	OpenAI-compat, then Gemini
Follow-up email draft	Local 7-8B	Never forced; button to regenerate on cloud	OpenAI-compat
Embeddings (indexing + search)	nomic-embed-text via Ollama	Never in v1	n/a

Table 5 - Routing policy per AI task (defaults, editable)

The economics deserve their own figure, because they are the quiet argument for the whole project. The chart below compares the estimated marginal cost of processing one 30-minute call across routing strategies, using public list prices as of October 2026 and the assumptions in Table 6. The local stack costs about a cent of electricity. The hybrid strategy - local STT with one cloud deep-analysis pass - costs roughly four cents. Even the most profligate all-cloud chain stays under a quarter. And the amber line at the top is what a Gong seat costs per call once its publicly reported fees are amortized: it is not in the same sport.

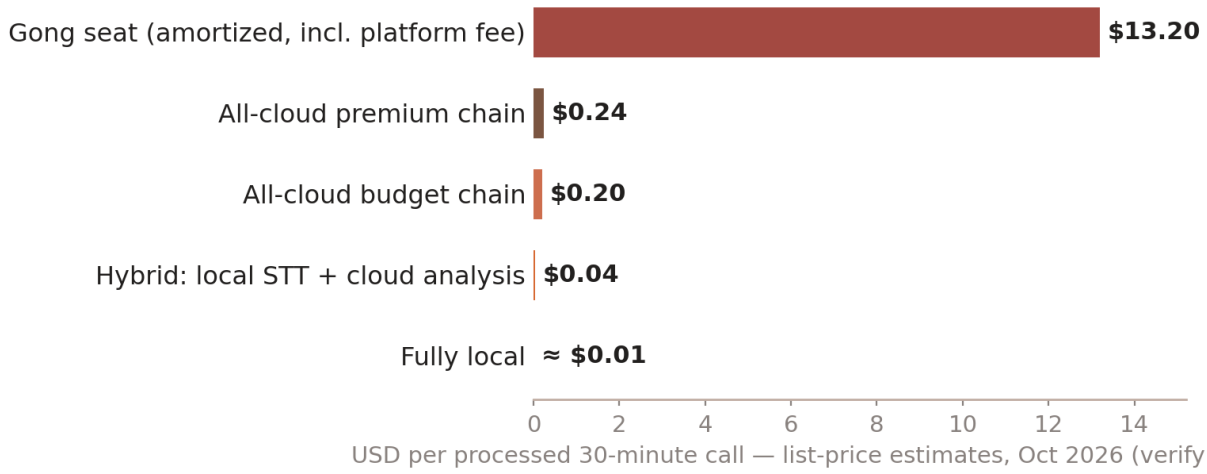


Figure 3 - Estimated marginal cost per processed 30-minute call by routing strategy (USD, list prices, Oct 2026). Gong figures derive from third-party pricing analyses; treat as estimates and verify.

Assumption	Value	Basis
30-minute call transcript	about 4,500 words, 6,000 tokens	Typical conversational speaking rate
Deep analysis call	8,000 tokens in, 1,500 out	Transcript + rubric + few-shot exemplars
Whisper cloud STT	$0.006 per minute	Public list price; verify at build time
Analysis-tier LLM pricing	$2.50 / $10 per 1M tokens in / out	GPT-4o-class public list; Gemini-flash and Sonnet-class priced proportionally
Local run	about $0.01 electricity per call	Desktop-class machine, no per-token charge
Gong comparison	$1,600 seat + $5,000 platform, 500 calls/yr	Publicly reported 2026 pricing analyses (Claap, Oliv, Outdoo); upper end of reported range

Table 6 - Cost model assumptions (all figures editable in Settings)

Model currency is a standing risk, so the engine treats model names as configuration, never as code. The catalog in the Appendix names the local model classes verified current as of October 2026 - the Qwen3 family around 8B, gpt-oss:20b for 16 GB machines, nomic-embed-text for embeddings - but the policy table stores whatever you pull. A one-line settings change re-points every task at a new model, and the run log tells you afterward whether quality moved. That is the entire upgrade story: AMBER never needs to be rebuilt to ride the next model wave.

5 · Data Model

The schema is designed around one reading: a call is a tree, and everything else in the system is either a branch of that tree or a lens over many trees. Transcripts belong to calls; insights belong to transcripts; deals and contacts are the entities the calls are about, and they collect calls over time. The design keeps the hot path - the call detail view - to a single query with eager relations, and it keeps the cold path - search - to a vector scan over transcript chunks. Below is the core of the Prisma schema for v1, trimmed to the essential fields; the full file lives in the repository scaffold this blueprint hands to the build phase.

```
model Deal {
  id            String   @id @default(cuid())
  title         String
  company       String?
  stage         String   @default("discovery")   // discovery..won/lost
  value         Float?
  currency      String   @default("EUR")
  meddic        Json?    // metrics, economicBuyer, decisionCriteria, ...
  riskFlags     Json?    // [{code, severity, note, callId?}]
  nextStep      String?
  nextStepDate  DateTime?
  contacts      Contact[]
  calls          Call[]
  createdAt     DateTime @default(now())
  updatedAt     DateTime @updatedAt
}

model Call {
  id           String   @id @default(cuid())
  dealId       String?
  deal         Deal?    @relation(fields: [dealId], references: [id])
  title        String
  occurredAt   DateTime @default(now())
  mediaHash    String   @unique          // SHA-256, de-dup key
  audioPath    String                    // media vault, content-addressed
  durationSec  Int
  status       String   @default("queued") // queued|normalizing|transcribing|analyzing|done|failed
  stage        Int      @default(0)        // resumable pipeline cursor
  transcript   Transcript?
  insights     Insight[]
  scorecard    Scorecard?
  notes        Note[]
  runs         ProviderRun[]              // telemetry per AI call
}

model Transcript {
  id        String    @id @default(cuid())
  callId    String    @unique
  call      Call      @relation(fields: [callId], references: [id])
  language  String    @default("en")
  text      String                     // full plain text
  segments  Segment[]
  embeddingIds Json?                  // vector chunk ids
}

model Segment {
  id        String    @id @default(cuid())
  transcriptId String
  speaker   String                    // "you" | "prospect" | "other" | learned name
```

```
  startMs   Int
  endMs     Int
  text      String
}

model Insight {
  id        String    @id @default(cuid())
  callId    String
  kind      String    // topic|objection|sentiment|meddic|risk|question|keyword
  payload   Json      // typed per kind
  confidence Float
  span      Json?     // [startMs, endMs] when locatable
}

model Scorecard {
  id       String  @id @default(cuid())
  callId   String  @unique
  rubric   Json    // criterion -> {score, weight, evidence}
  total    Float
}
```

Schema 1 - Core Prisma models (v1, trimmed)

A few modeling decisions earn their explanation. Insight rows are a single polymorphic table keyed by kind rather than one table per insight type; personal scale never stresses this, and it keeps the analysis prompt contract - one JSON document per call - mapped trivially to storage. ProviderRun rows exist for every AI invocation with provider, model, latency, token counts, and estimated cost; they are what turn the cost chart from an estimate into a measurement, and they are the only telemetry AMBER keeps, entirely on your disk. Embeddings are stored as BLOB columns with chunk text and call reference; retrieval is brute-force cosine over them because at hundreds of calls the scan is milliseconds and adds zero dependencies.

Entity	Purpose	Notable fields
Deal	The opportunity a set of calls advances	stage, value, meddic Json, riskFlags, nextStep + date
Contact	Person on the other side	name, role, email; links deal and calls
Call	One recorded conversation	mediaHash (unique), status, stage cursor for resumability
Transcript / Segment	The words, with speaker turns	segments with startMs/endMs per line
Insight	One extracted finding	kind, payload, confidence, optional time span
Scorecard	Rubric evaluation for one call	rubric Json with per-criterion evidence
Note / FollowUpDraft	Human-editable outputs	editable text; drafts regenerate on demand
ProviderRun	Telemetry per AI invocation	provider, model, latency, tokens, cost estimate
Setting	Policy table and model config	JSON blobs per task; never secrets

Table 7 - Core entities and their purpose

Deletion is designed, not bolted on. Deleting a call cascades to its transcript tree, insights, scorecard, and vault file; deleting a deal keeps calls but detaches them, because a conversation happened even if the opportunity died. A prospect's data can be purged on request by deleting their contact and cascading - and the retention chapter specifies why, in the EU, you should assume that duty applies to you. Backups are one SQLite file plus the vault directory; copying both is a complete, restorable AMBER.

6 · Feature Specifications - Four Pillars

This chapter is the build contract for v1: each pillar is specified as a user flow, the AI tasks it consumes (all defined in Table 5), the interface it renders, and the MVP line where later work starts. The through-line for every pillar is the same discipline: deterministic computation wherever arithmetic suffices, model judgment only where language understanding is genuinely required, and every model output carrying a confidence you can see.

Pillar 1 - The Core Loop

The loop begins at the Upload Inbox: drag a recording in, and AMBER computes its hash, checks it against the vault for de-duplication, asks (when filename heuristics fail) which deal and contact to attach it to, and enqueues processing. Progress renders as a staged indicator - normalizing, transcribing, analyzing, done - each stage persisting its output so an interrupted run resumes rather than restarts. When the call completes, you land on the Call Detail view: the transcript on the left with speaker turns, timestamps, and clickable highlights; the metric strip above it; and tabs for insights, notes, and the follow-up draft on the right.

The metric strip is the part that must never lie, so none of it is model-generated. Talk-listen ratio, longest monologue, questions asked, words per minute, and per-speaker speaking time are all computed from Segment timings - pure arithmetic, reproducible to the millisecond, immune to model drift. What the models produce sits beneath the strip, clearly labeled: topics with spans, objections with the exact sentences that raised them, sentiment per span rather than a misleading whole-call average, and keywords with counts. Every insight chip links to its evidence span in the transcript; tapping it seeks the audio to that moment, because the point of analysis is to hear the moment again.

The notes and follow-up draft close the loop. Meeting notes are generated from the transcript with a prompt you can read and edit in Settings; they render as an editable document with a copy button and a plain .md export. The follow-up email draft is persona-aware: it references the prospect's stated problems, the commitments made, and the next step with its date, and it regenerates with one click - on local by default, on cloud by choice. In v1 these drafts are the end of the automation; auto-sending anything is deliberately out of scope, because the one-person version of AMBER should draft, not act.

Pillar 2 - Deal Intelligence

Deal intelligence turns calls into a living pipeline. The MEDDIC extraction runs on every call linked to a deal and updates the deal's fields: metrics the prospect volunteered, economic buyer (named or suspiciously absent), decision criteria and process, identified pain, and champion (named or absent). Because extraction is structured JSON with confidence, the deal view shows not just the value but the evidence - each field links to the span in the call where it was learned, and stale fields (nothing new in 14 days) fade visually. The pipeline board is a local kanban over stages you define; drag-and-drop updates stage, and wins/losses write a closure note automatically.

Risk flags are rules plus inference, in that order. Deterministic rules fire first: no next step set within 24 hours of a call ends; a next step date that passed without a following call; single-threaded - every call with the same one contact; budget never discussed across all calls on a deal; two consecutive calls with declining sentiment. The model layer then writes risk notes attached to spans (a hesitation pattern, a competitor mention without a rebuttal). Flags render on the board as severity chips, and each flag links to the exact calls that raised it. The deal timeline at the bottom of the deal view is auto-built: one row per call and per stage change, newest first - the story of the deal, assembled without anyone writing it.

Pillar 3 - Coaching

Coaching in a one-person system means comparing you to you. The scorecard rubric is fully user-owned: a default set ships - next step set, question rate above target, talk ratio within your band, objections answered rather than deflected, discovery before pitch - and every criterion carries a weight you set. Scoring a call runs the rubric prompt with the transcript and writes per-criterion evidence links, so "question rate: 6, below target 10" comes with the six questions highlighted in place. The rubric edit screen is where coaching strategy lives; AMBER never hides the rubric the way closed products do.

Trends render as small multiples over the metric strip history: scorecard total, talk ratio, question rate, objection-handling rate, all on a per-week basis with a target band you set per metric. The best-practice library is the compounding asset: save any transcript span as a snippet with a tag (objection handled well, discovery sequence, pricing defense), and the library indexes it by embedding so "how did I handle the budget pushback last quarter" retrieves your own moments, not a generic coach's. Gong needs a team for this feature to make sense; a single user with a growing library gets more from it, not less.

Pillar 4 - The Library

The library is search over everything you have ever said on a call. Semantic search runs the query through the same embedding model as indexing (local, always) and returns transcript chunks ranked by cosine similarity, each result showing the call, the date, the linked deal, and the matched span with the query terms effectively highlighted by proximity. Filters compose: by date range, by deal, by contact, by insight kind (only moments flagged as objections), by sentiment span, and by scorecard score band. Saved searches pin to the sidebar - "lost deals, pricing objections" is one click away forever.

The MVP line for the library is deliberately conservative: no cross-call theme clustering, no auto-generated topic taxonomy, no "ask my history anything" chat agent in v1. Those are attractive and all buildable on the same vectors later; they are deferred because the retrieval core must first prove itself in daily use. The test the library must pass is simple: three months in, when you wonder how a phrase landed six weeks ago, the answer arrives before the memory of asking the question fades. That is the whole feature; everything beyond it is ornament.

7 · Design System - Warm Calm

The visual direction you selected - Warm Calm, in the family of Arc and Cron - is a strategy, not a mood. Revenue tools are built to make operators anxious: dense dark dashboards, urgency reds, alert bells. AMBER inverts that. You will spend real time reading your own conversations here, often at night, often right after a call went badly. The interface should feel like paper and lamplight: warm neutrals, generous rounding, unhurried motion, and color reserved for meaning. Calm is also a legibility strategy - a page that does not shout can afford to say more.

Token	Value	Usage
bg / paper	#FAF7F2	App canvas; the "cream paper" base
bg / raised	#F3EDE5	Cards, sheets, the inbox surface
ink / primary	#2A2520	Body text and headings; never pure black
ink / secondary	#6E665D	Metadata, timestamps, disabled states
accent / amber	#C96F3B	Primary actions, active states, links; the only loud color
accent / deep	#8A5A2E	Hover and pressed states of amber
success / warn / risk	#5C8A64 / #C2A24B / #B0574D	Muted, desaturated; risk red never saturates
line / hairline	#E7DFD4	1px borders; separation without weight
radius	12 / 16 / 24 px	Controls / cards / sheets; rounding is the warmth carrier
type / display	Fraunces 600-700	Page titles, deal names, the "editorial" voice
type / UI + body	Inter 400-600	Interface text and transcripts; tabular numerals for all metrics
mono / data	JetBrains Mono	Timestamps, hashes, token counts
motion	150-250 ms, ease-out	Hover lifts 1px, sheets slide 12px; nothing bounces

Table 8 - Warm Calm core tokens (app design system)

Two rules carry most of the aesthetic weight. First, amber is rationed: it marks interactive things and active things, nothing else - a page at rest is almost entirely warm neutral, so a single amber element reads as an invitation. Second, numbers are always tabular: metrics, timings, and scores render in tabular numerals so columns of figures align like set type. Both rules exist because the transcript view is the product; it must survive hours of reading without fatigue, and everything else in the system yields to that priority.

- CallCard - inbox and library rows: date, deal, duration, score chip, three-line transcript preview.
- TranscriptSegment - speaker chip, timestamp (mono), text; active highlight in amber at 8% tint.
- InsightChip - kind-colored at low saturation, always links to its evidence span.
- ScoreRing - the scorecard total as a ring, rubric detail one tap away.
- PipelineBoard - columns of DealCards; risk chips summarize each deal's open flags.
- StatStrip - the deterministic metrics band; every figure tabular, every figure auditable by click.

Accessibility is part of calm, not in tension with it. All text meets 4.5:1 contrast on the paper tones (the ink pair was chosen to clear it with margin); focus rings are 2px amber and never removed; transcript text sits at 16px minimum with 1.6 line height. A dark theme is deferred, not rejected - a warm dark done well is a real project, and shipping a bad one early would betray the whole direction. Internationalization is structural (all strings externalized from day one) but v1 ships English only.

8 · Security, Privacy and Legal

AMBER's privacy posture is a consequence of its architecture, not a policy bolted on. Audio and video files never leave the machine that runs AMBER; by default every model that touches a call runs locally, and when a task escalates to a cloud provider, what crosses the network is text - a transcript excerpt, not a recording - with an optional redaction pass applied first. API keys live in the OS keychain (or a .env file git-ignored by the scaffold) and never in the database, never in logs, never in telemetry - and there is no telemetry: no crash reporter, no analytics, no "phone home" of any kind, because a personal tool has nobody to report to.

- Keys: OS keychain or .env; the Settings UI writes them there and displays only masked values.
- Redaction: configurable patterns (names, emails, phones, custom regex) applied before any cloud call; the redaction map stays local so insights re-identify on your screen only.
- Budget: a monthly cloud spend cap in Settings; the router refuses escalation when the cap is hit, and the refusal is visible in the run log.
- At rest: the SQLite file and vault inherit full-disk encryption (FileVault / BitLocker); SQLCipher is a documented upgrade if you want the database itself encrypted.
- Backups: one file plus one folder; copy both and you hold the entire system.

The legal section is short and serious, and it is not legal advice. You are in the EU, and two regimes matter. Recording consent: many EU jurisdictions require the knowledge or consent of participants for recording; the safest durable practice is to state at the top of a call that it is being recorded - most meeting platforms can display a standing notice, and honesty ages better than cleverness. GDPR: your transcripts contain other people's personal data. That gives you duties in practice - purpose limitation (your own coaching), retention limits you actually enforce, and deletion on request: the data model's contact-cascade delete exists precisely so "please delete everything about me" is one command, honored honestly.

Treat the following as the compliance floor for personal use: keep recordings only as long as they serve your coaching and deal work; when a relationship ends, delete the contact cascade; never move raw audio to a cloud provider, even a trusted one, unless you have decided the trade-off consciously; and if you ever share a transcript outside AMBER, share it redacted. The system is built so that the private-by-default path is also the path of least resistance - that alignment between convenience and correctness is the entire security design.

9 · Build Roadmap and Risk Register

The roadmap is phrased in focused sessions - long, undistracted working blocks of roughly three hours - because for a personal project, calendar time is a fiction and attention is the real currency. The sequence below is dependency-ordered, not priority-ordered: each phase produces something usable on its own, so the project can stop after any phase and what exists is already worth having. The estimates assume the blueprint in hand (this document), which is the point of writing it first.

Phase	Scope	Exit criteria	Effort
P0 Foundations	Scaffold, Prisma schema, vault, upload + hash + queue, pipeline skeleton with resumable stages	A file lands in the vault, a call row walks stages, restart resumes mid-flight	3-4 sessions
P1 Core loop	whisper.cpp wiring, transcript view with speakers and timestamps, deterministic metric strip	A 30-min call produces a readable transcript and honest metrics, fully offline	4-5 sessions
P2 Intelligence	Provider registry + five adapters, policy table, router with escalation, insight prompts + validation	Insights with evidence spans render; run log records cost and latency	4-6 sessions
P3 Deal + Library	MEDDIC extraction, risk rules, board, deal timeline; embeddings + semantic search + filters	A query retrieves the right span across all calls; board shows live flags	4-5 sessions
P4 Coaching	Rubric screen, scorecard generation, trend charts, best-practice snippets	Scorecards accumulate; trends answer "am I improving" honestly	3-4 sessions
P5 Polish	Notes + follow-up drafts, saved searches, exports (.md/.eml), Warm Calm pass over every surface	The whole loop feels deliberate; nothing requires a terminal	3-4 sessions

Table 9 - Phase plan with exit criteria

Twenty to twenty-eight sessions for the full v1 is the honest envelope; the midpoint is roughly three months of focused weekends. The risk register below names the things most likely to bend that line, with mitigations that are already reflected in the architecture rather than promises to be careful.

Risk	Likelihood	Impact	Mitigation
Local diarization quality disappoints on 3+ speaker calls	High	Medium	Ships as heuristic + LLM relabel; WhisperX-style VAD + forced alignment is the documented upgrade path (sidecar)
CPU-only STT too slow on long calls	Medium	Medium	Policy defaults to small-class models on CPU; distil-class cuts time ~6x at ~1% WER cost; GPU machines auto-select large-class
Model drift: catalog names age quickly	Certain	Low	Models are configuration, not code; Appendix catalog is dated and the run log verifies quality after any swap
Structured outputs fail schema too often locally	Medium	Medium	Validation + one retry, then escalation (Table 5); prompt registry with few-shot exemplars tuned per task
Scope creep - the personal-project classic	High	High	This blueprint is the contract; new ideas enter a v2 list, never the current phase
Cloud API cost surprises	Low	Low	Monthly cap in Settings; run log makes spend visible per call; local-first means cloud is opt-in by construction
Consent/GDPR exposure from recordings	Low	High	Standing record notice; contact-cascade deletion; redaction before any cloud text leaves the machine

Table 10 - Risk register

One meta-risk deserves its own sentence: the temptation to build infrastructure before the loop. Every phase above produces user-visible value, and the sequence deliberately puts transcription and the metric strip before anything intelligent, because the moment AMBER shows you your own call honestly is the moment it earns the rest of the work.

10 · Appendix - Hardware, Models, Costs, Glossary

This appendix holds the reference tables the build phase will reach for, plus the glossary for future-you reading this document after the stack has inevitably moved on. All model names reflect the Ollama library and public cloud tiers as of October 2026; the model-currency rule in Chapter 4 - models are configuration - is what keeps this appendix advisory rather than load-bearing.

Tier	Specification	What to expect
Minimum	16 GB RAM, modern 6-core CPU, any recent iGPU	small-class STT near real time after processing; 7-8B local LLM at usable speeds; embeddings trivial
Recommended	32 GB RAM, 8 GB+ VRAM GPU (RTX-class or Apple Silicon unified 24 GB+)	large-class STT faster than real time; deep-analysis prompts run locally with headroom for longer calls
CPU-only note	distil-class STT models	About 6x faster than large at roughly 1% word-error cost - the right default when no GPU is present

Table 11 - Hardware tiers

Task	Local default class	Cloud fallback class
General LLM (analysis, notes, relabel)	Qwen3 family 7-8B; gpt-oss:20b on 16 GB machines (128K context)	GPT-4o-class / Sonnet-class (200K) / Gemini 2.x-class
Speech-to-text	Whisper small-or-better via whisper.cpp; faster-whisper large-v3 on GPU	Whisper API (text-only, never audio, by policy)
Embeddings	nomic-embed-text (the library default, 274 MB); mxbai-embed-large if quality disappoints	None - embeddings never leave the machine in v1
Structured extraction	Same as general LLM, JSON-schema constrained	Same chain as analysis; Anthropic preferred for long transcripts

Table 12 - Model catalog by task (October 2026)

The cost figure worth internalizing is not any single row of Table 6 - it is the shape: local marginal cost is effectively zero, cloud marginal cost is cents, and both are so far below any seat-based commercial product that cost should never be the reason a task runs locally; privacy and ownership should be, and are. Verify every list price at build time; the assumptions table is one settings screen, and the run log will replace estimates with measurements within your first ten calls.

- STT (speech-to-text) - converting recorded audio to timestamped text.
- Diarization - attributing text spans to speakers ("who said what").
- MEDDIC - a qualification framework: Metrics, Economic buyer, Decision criteria/process, Identify pain, Champion.
- Embedding / vector - a numeric representation of text enabling semantic (meaning-based) search.
- RTF (real-time factor) - processing time divided by audio duration; below 1.0 means faster than real time.
- Structured output - model output constrained to a JSON schema the code can validate.

- Redaction - stripping identifiers (names, emails, phones, custom patterns) before text leaves the machine.

Final word to the builder, from the team that wrote this: keep the loop honest. The metric strip must stay deterministic; the prompts must stay visible; the router must stay local-first; the design must stay warm. Everything else in this blueprint is negotiable with experience - those four properties are the product's character, and they are the reason a personal system can honestly call itself better rather than merely cheaper.